Лабораторная работа № 1 Создание сетевого серверного приложения

Цель работы

Изучение принципов организации обмена данными по сети и создание сетевого серверного приложения.

Теоретическая информация

Введение в сети и протоколы

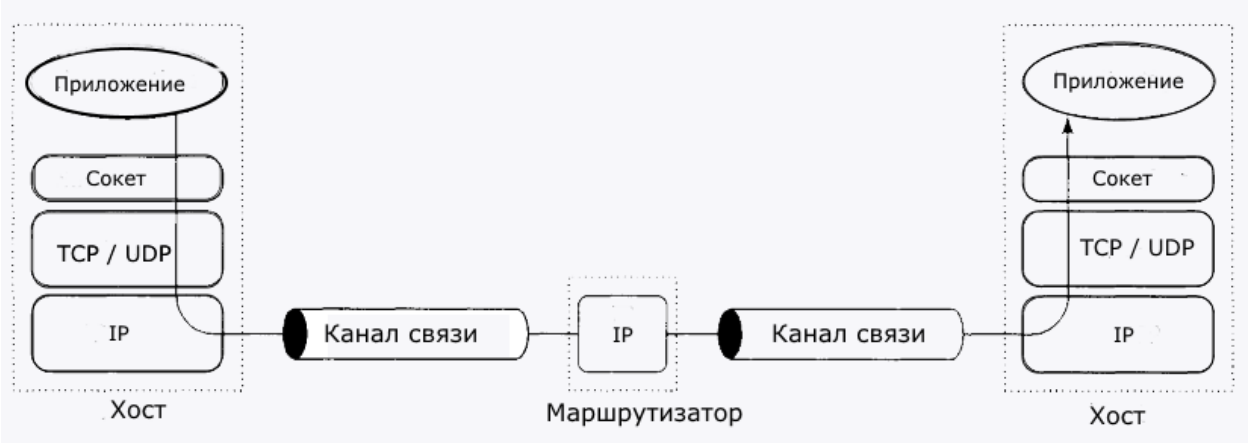
Сегодня миллионы компьютеров и устройств связаны в глобальную сеть интернет, либо в отдельные локальные подсети. В связи с этим возникает необходимость создания приложений, которые бы использовали все преимущества передачи данных по сети. Например, одним из распространенных приложений, которое использует передачу по сети, является веб-браузер. И платформа .NET и язык программирования C# предоставляют все необходимые возможности для создания приложений, которые могут взаимодействовать по сети и использовать различные сетевые протоколы.

Но прежде чем переходить непосредственно к созданию приложений, надо пару слов сказать, что вообще представляет собой коммуникация в сети.

Вся сеть состоит из отдельных элементов - хостов, которые представляют собой компьютеры и другие подключенные устройства. Между собой они соединены каналами связи (кабели Ethernet, Wi-Fi и т.д.) и маршрутизаторами. Маршрутизаторы объединяют компьютеры в подсети и контролируют передачу данных между ними.

Но компьютеры-хосты не взаимодействуют абы как между собой. Они применяют протоколы. Протокол представляет собой соглашения о том, как пакеты данных будут передаваться по каналам коммуникации. Таким образом, протокол упорядочивает взаимодействие.

Существует множество различных протоколов. Протоколы, которые используются для передачи данных по сети, составляют семейство протоколов TCP/IP. Основные из них: Internet Protocol (IP), Transmission Control Protocol (TCP) и User Datagram Protocol (UDP). Причем эти протоколы организованы в уровневую систему:



IP представляет сетевой уровень. Он использует нижележащие уровни, которые представляют физические каналы коммуникации - кабели Ethernet и т.д., для передачи пакетов с данными другому хосту.

Выше IP располагается транспортный уровень, который образуют протоколы TCP и UDP. Эти протоколы используют определенные порты для передачи данных. TCP позволяет отследить потерю пакетов и их дублирование при передаче. UDP подобного не позволяет сделать и нацелен на простую передачу данных.

Однако приложение взаимодействует с уровнем TCP / UDP не напрямую, а через специальный API, который предоставляют сокеты. Сокеты - это не какой-либо протокол, это просто интерфейс для создания сетевых приложений, который опирается на встроенные возможности операционной системы.

В зависимости от используемого протокола различают два вида сокетов: потоковые сокеты (stream socket) и дейтаграммные сокеты (datagram socket). Потоковые сокеты используют протокол TCP, дейтаграммные - протокол UDP.

В итоге, когда приложение посылает сообщение приложению, запущенному на другом хосте, то приложение обращается к сокетам для передачи данных на уровень TCP / UDP. Далее с этого транспортного уровня данные передаются сетевому уровню - уровню протокола IP. И этот протокол передает данные далее физическим уровням, и после этого данные уходят по сети.

Чтобы уникально определять хосты в сети каждый хост имеет адрес. Существует несколько различных протоколов адресов. В настоящее время наиболее распространен протокол IPv4, который предполагает представление адреса в виде 32-битного числа, например, 37.120.16.63. Такой адрес содержит четыре числа, разделенных точками, и каждое число находится в диапазоне от 0 до 255. Однако также в последнее время набирает оборот использование адресов протокола IPv6, которые представляют собой 128-битное значение.

Однако такие адреса очень сложно запомнить, поэтому в реальности чаще оперируют доменами. Домены представляют специальные названия, используемые для интернет-адресов. Например, есть доменное имя "www.microsoft.com", ему соответствует адрес в формате IPv4 2.23.143.150. Но для протокола IP, через который идет взаимодействие, доменные адреса не

существуют. Поэтому при отправке или передаче данных по доменному имени, компьютер еще обращается к службам Domain Name System (DNS), который выполняют сопоставление между интернет-адресами в формате IPv4 или IPv6 и доменными названиями.

Кроме адреса при сетевых взаимодействиях используются порты. Порт представляет 16-битное число в диапазоне от 1 до 65 535. Использование портов позволяет разграничить несколько запущенных приложений на одном хосте.

Собственно, это все базовые принципы взаимодействия по сети, которые надо знать. В реальности, как правило, при создании приложений не потребуется глубокого знания всех протоколов и нюансов их работы. Если в редких случаях возникнет необходимость более детального знания протоколов, то в этом случае можно обратиться к специализированной литературе, например, к книгам Олифера или Танненбаума.

Адреса в .NET

Прежде чем отправить запрос к какому-нибудь ресурсу, компьютер обращается к DNS-северу, чтобы по имени ресурса получить его ip-адрес. И затем уже обращается по этому ip-адресу.

Все ip-адреса представляют 32-битное (протокол IPv4) или 128-битное значение (протокол IPv6), например, 31.170.165.181.

В системе классов .NET ip-адрес представлен классом **IPAddress**. Этот класс позволяет управлять адресами с помощью следующих свойств и методов:

Метод Parse(): преобразует строковое представление адреса в IPAddress

```
IPAddress ip = IPAddress.Parse("127.0.0.1"); // ip указывает на локальный адрес
```

- Статическое свойство Loopback: возвращает объект IPAddress для адреса 127.0.0.1. Аналогично вышеприведенному коду
- Статическое свойство Any: возвращает объект IPAddress для адреса 0.0.0.0
- Статическое свойство Broadcast: возвращает объект IPAddress для адреса 255.255.255.255
- Также для получения адреса в сети используется класс **IPHostEntry**. Он содержит информацию об определенном компьютере-хосте.

С помощью свойства **HostName** этот класс возвращает имя хоста, а с помощью свойства **AddressList** - все ip-адреса хоста, так как один компьютер может иметь в сети несколько ip-адресов.

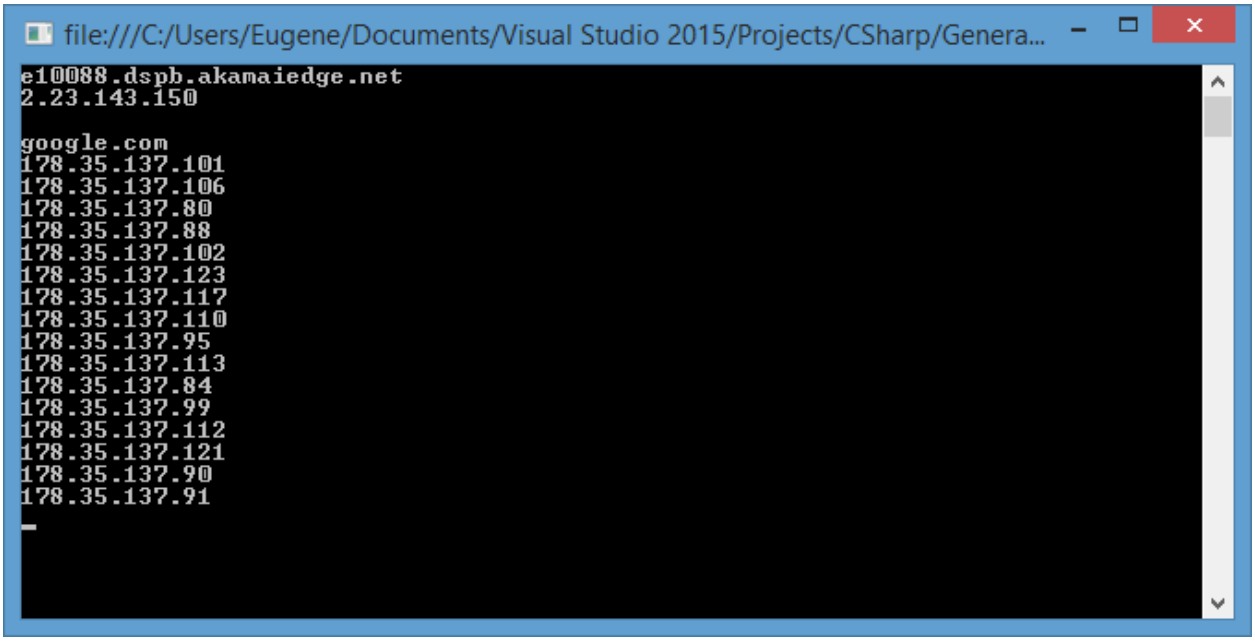
Для взаимодействия с DNS-сервером и получения ip-адреса применяется класс Dns. Для получения информации о хосте компьютера и его адресах у него имеется метод **GetHostEntry()**:

```
IPHostEntry host1 = Dns.GetHostEntry("www.microsoft.com");
Console.WriteLine(host1.HostName);
foreach (IPAddress ip in host1.AddressList)
    Console.WriteLine(ip.ToString());

Console.WriteLine();

IPHostEntry host2 = Dns.GetHostEntry("google.com");
Console.WriteLine(host2.HostName);
foreach (IPAddress ip in host2.AddressList)

    Console.WriteLine(ip.ToString());
```



Класс Socket

В основе межсетевых взаимодействий по протоколам TCP и UDP лежат сокеты. В .NET сокеты представлены классом System.NET.Sockets.Socket, который предоставляет низкоуровневый интерфейс для приема и отправки сообщений по сети.

Рассмотрим основные свойства данного класса:

- `AddressFamily`: возвращает все адреса, используемые сокетом. Данное свойство представляет одно из значений, определенных в одноименном перечислении **AddressFamily**. Перечисление содержит 18 различных значений, наиболее используемые:
  - `InterNetwork`: адрес по протоколу IPv4
  - `InterNetworkV6`: адрес по протоколу IPv6
  - `Ipx`: адрес IPX или SPX
  - `NetBios`: адрес NetBios
- `Available`: возвращает объем данных, которые доступны для чтения
- `Connected`: возвращает true, если сокет подключен к удаленному хосту

- `LocalEndPoint`: возвращает локальную точку, по которой запущен сокет и по которой он принимает данные
- `ProtocolType`: возвращает одно из значений перечисления ProtocolType, представляющее используемый сокетом протокол. Есть следующие возможные значения:
    - `Ggp`
    - `Icmp`
    - `IcmpV6`
    - `Idp`
    - `Igmp`
    - `IP`
    - `IPSecAuthenticationHeader` (Заголовок IPv6 AH)
    - `IPSecEncapsulatingSecurityPayload` (Заголовок IPv6 ESP)
    - `IPv4`
    - `IPv6`
    - `IPv6DestinationOptions` (Заголовок IPv6 Destination Options)
    - `IPv6FragmentHeader` (Заголовок IPv6 Fragment)
    - `IPv6HopByHopOptions` (Заголовок IPv6 Hop by Hop Options)
    - `IPv6NoNextHeader` (Заголовок IPv6 No next)
    - `IPv6RoutingHeader` (Заголовок IPv6 Routing)
    - `Ipx`
    - `ND`
    - `Pup`
    - `Raw`
    - `Spx`
    - `SpxII`
    - `Tcp`
    - `Udp`
    - `Unknown` (неизвестный протокол)
    - `Unspecified` (неуказанный протокол)

Каждое значение представляет соответствующий протокол, но наиболее используемыми являются Tcp и Udp.

- `RemoteEndPoint`: возвращает адрес удаленного хоста, к которому подключен сокет
- `SocketType`: возвращает тип сокета. Представляет одно из значений из перечисления **SocketType**:
    - `Dgram`: сокет будет получать и отправлять дейтаграммы по протоколу `Udp`. Данный тип сокета работает в связке с типом протокола - `Udp` и значением `AddressFamily.InterNetwork`
    - `Raw`: сокет имеет доступ к нижележащему протоколу транспортного уровня и может использовать для передачи сообщений такие протоколы, как `ICMP` и `IGMP`

- Rdm: сокет может взаимодействовать с удаленными хостами без установки постоянного подключения. В случае, если отправленные сокетом сообщения невозможно доставить, то сокет получит об этом уведомление
- Seqpacket: обеспечивает надежную двустороннюю передачу данных с установкой постоянного подключения
- Stream: обеспечивает надежную двустороннюю передачу данных с установкой постоянного подключения. Для связи используется протокол TCP, поэтому этот тип сокета используется в паре с типом протокола Tcp и значением AddressFamily.InterNetwork
- Unknown: адрес NetBios

Для создания объекта сокета можно использовать один из его конструкторов. Например, сокет, использующий протокол Tcp:

```
Socket socket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
```

Или сокет, использующий протокол Udp:

```
Socket socket = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
```

Таким образом, при создании сокета мы можем указывать разные комбинации протоколов, типов сокета, значений из перечисления AddressFamily. Однако в то же время не все комбинации являются корректными. Так, для работы через протокол Tcp, нам надо обязательно указать параметры: AddressFamily.InterNetwork, SocketType.Stream и ProtocolType.Tcp. Для Udp набор параметров будет другим: AddressFamily.InterNetwork, SocketType.Dgram и ProtocolType.Udp. Для других протоколов набор значений будет отличаться. Поэтому использование сокетов может потребовать некоторого знания принципов работы отдельных протоколов. Хотя в отношении Tcp и Udp все относительно просто.

Общий принцип работы сокетов

При работе с сокетами вне зависимости от выбранных протоколов мы будем опираться на методы класса Socket:

- Accept() : создает новый объект Socket для обработки входящего подключения
- Bind() : связывает объект Socket с локальной конечной точкой
- Close() : закрывает сокет
- Connect() : устанавливает соединение с удаленным хостом
- Listen() : начинает прослушивание входящих запросов

- `Poll()`: определяет состояние сокета
- `Receive()`: получает данные
- `Send()`: отправляет данные
- `Shutdown()`: блокирует на сокете прием и отправку данных

В зависимости от применяемого протокола (`TCP`, `UDP` и т.д.) общий принцип работы с сокетами будет немного различаться.

При применении протокола, который требует установление соединения, например, `TCP`, сервер должен вызвать метод `Bind` для установки точки для прослушивания входящих подключений и затем запустить прослушивание подключений с помощью метода `Listen`. Далее с помощью метода `Accept` можно получить входящие запросы на подключение в виде объекта `Socket`, который используется для взаимодействия с удаленным узла. У полученного объекта `Socket` вызываются методы `Send` и `Receive` соответственно для отправки и получения данных. Если необходимо подключиться к серверу, то вызывается метод `Connect`. Для обмена данными с сервером также применяются методы `Send` или `Receive`.

Если применяется протокол, для которого не требуется установление соединения, например, `UDP`, то после вызова метода `Bind` не надо вызывать метод `Listen`. И в этом случае для приема данных используется метод `ReceiveFrom`, а для отправки данных - метод `SendTo`.

Создание серверного приложения на сокетах TCP

Рассмотрим, как создать сервер, работающий по протоколу TCP, с помощью сокетов. Общая схема работы серверного сокета TCP будет следующей:

Код программы сервера будет таким:

```
using System;
using System.Text;
using System.Net;
using System.Net.Sockets;

namespace SocketTcpServer
{
    class Program
    {
        static int port = 8005; // порт для приема входящих запросов
        static void Main(string[] args)
        {
            // получаем адреса для запуска сокета
            IPEndPoint ipPoint = new
IPEndPoint(IPAddress.Parse("127.0.0.1"), port);

            // создаем сокет
            Socket listenSocket = new
Socket(AddressFamily.InterNetwork,
                                                SocketType.Stream,
ProtocolType.Tcp);
            try
            {
                // связываем сокет с локальной точкой, по которой будем
принимать
```

```
                    данные
                listenSocket.Bind(ipPoint);

                // начинаем прослушивание
                listenSocket.Listen(10);

                Console.WriteLine("Сервер запущен. Ожидание
подключений...");

                while (true)
                {
                    Socket handler = listenSocket.Accept();
                    // получаем сообщение
                    StringBuilder builder = new StringBuilder();
                    int bytes = 0; // количество полученных байтов
                    byte[] data = new byte[256]; // буфер для
получаемых данных

                    do
                    {
                        bytes = handler.Receive(data);
                        builder.Append(Encoding.Unicode.GetString(data,
0, bytes));
                    }
                    while (handler.Available>0);

                    Console.WriteLine(DateTime.Now.ToShortTimeString()
+ ": " +
                                      builder.ToString());

                    // отправляем ответ
                    string message = "ваше сообщение доставлено";
                    data = Encoding.Unicode.GetBytes(message);
                    handler.Send(data);
                    // закрываем сокет
                    handler.Shutdown(SocketShutdown.Both);
                    handler.Close();
                }
            }
            catch(Exception ex)
            {
                Console.WriteLine(ex.Message);
            }
        }
    }
}
```

Вначале после создания сокета связываем его с локальной точкой методом `Bind`:

```
listenSocket.Bind(ipPoint);
```

Сокет будет прослушивать подключения по 8005 порту на локальном адресе 127.0.0.1. То есть клиент должен будет подключаться к локальному адресу и порту 8005.

Далее начинаем прослушивание:

```
listenSocket.Listen(10);
```

Метод `Listen` вызывается только после метода `Bind`. В качестве параметра он принимает количество входящих подключений, которые могут быть поставлены в очередь сокета.

После вызова метода `Listen` начинается прослушивание входящих подключений, и если подключения приходят на сокет, то их можно получить с помощью метода `Accept`:

```
Socket handler = listenSocket.Accept();
```

Метод `Accept` извлекает из очереди ожидающих запрос первый запрос и создает для его обработки объект `Socket`. Если очередь запросов пуста, то метод `Accept` блокирует вызывающий поток до появления нового подключения.

Для обработки запроса сначала в цикле `do..while` получаем данные с помощью метода `Receive`:

```
do
{
    bytes = handler.Receive(data);
    builder.Append(Encoding.Unicode.GetString(data, 0, bytes));
}
while (handler.Available > 0);
```

Метод `Receive` в качестве параметра принимает массив байтов, в который считываются полученные данные, и возвращает количество полученных байтов.

Если отсутствуют данные, доступные для чтения, метод `Receive` блокирует вызывающий поток до тех пор, пока не станут доступны данные, если не было установлено значение тайм-аута путем использования объекта `Socket.ReceiveTimeout`. Если значение тайм-аута было превышено, объект `Receive` выдаст исключение `SocketException`. Чтобы отследить наличие данных в потоке, используется свойство `Available`.

После получения данных клиенту посылается ответное сообщение с помощью метода `Send`, который в качестве параметра принимает массив байтов:

```
handler.Send(data);
```

В конце обработки запроса надо закрыть связанный с ним сокет:

```
handler.Shutdown(SocketShutdown.Both);

handler.Close();
```

Вызов метода `Shutdown()` перед закрытием сокета гарантирует, что не останется никаких необработанных данных. Этот метод в качестве параметра принимает значение из перечисления `SocketShutdown`:

- `Both`: остановка как отправки, так и получения данных сокетом
- `Receive`: остановка получения данных
- `Send`: остановка отправки данных

Запустим серверное приложение и убедимся в его работоспособности.

Консоль сервера:

```
Сервер запущен. Ожидание подключений...
```